

May 2026

GENERATING SOURCE CODE FROM UML DIAGRAMS

Comenius University Bratislava

Faculty of Mathematics, Physics and Informatics

Student: bc. Jahonoro Tojjeva

Supervisor: doc. Ing. Ivan Polášek, PhD.

Study programme: Applied Computer Science

OUR CONTENTS

01. Motivation and Goals

02. Related Work

03. UML Datasets

04. Our Approach

05. System Architecture

06. Current Progress

07. Experimental Results

08. Future Work

Motivation and Goals

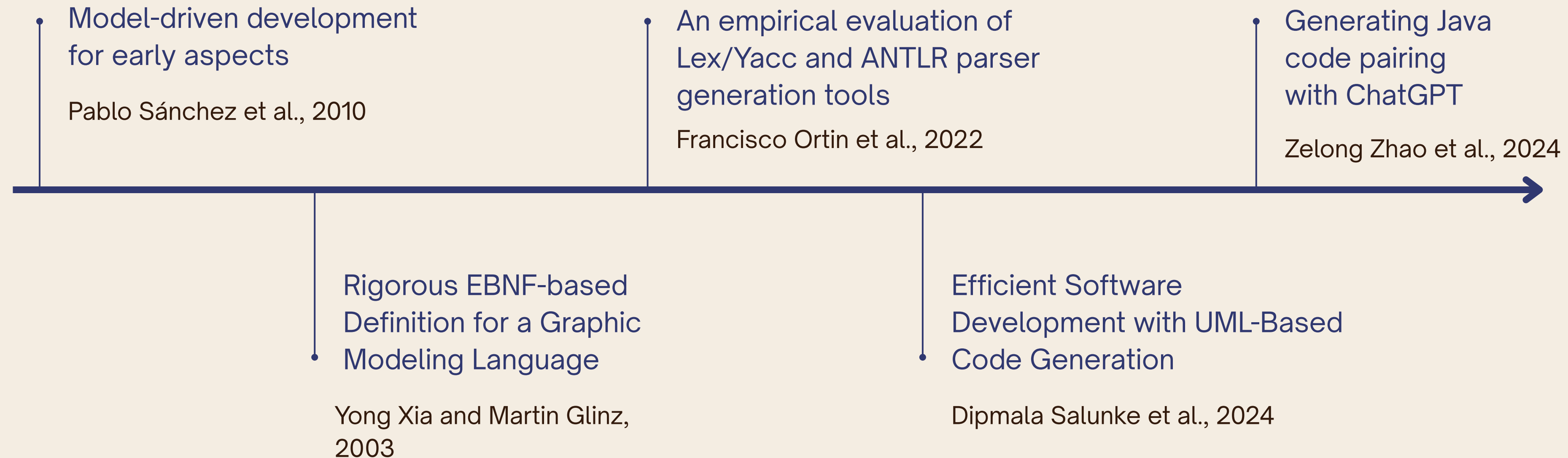
- 01 Large software systems are commonly designed using UML diagrams during early development phases
- 02 UML sequence diagrams help represent interactions between system components and business logic
- 03 Manual transformation from UML diagrams to source code is time-consuming and error-prone
- 04 Existing CASE tools often provide limited support for automated code generation from behavioral diagrams
- 05 The thesis focuses on automatic source code generation from UML diagrams using procedural and grammar-based parsing approaches
- 06 The work also investigates future comparison with AI-assisted code generation approaches based on Large Language Models (LLMs)

THESIS OBJECTIVES

Analyze the possibilities of generating source code from diagrams.

Create a prototype as a plugin or tool for (e.g.) draw.io to generate source code from a UML sequence diagram. Compare the results with the possibility of generating via LLM.

RELATED WORK AND SCIENTIFIC PAPERS



MODEL-DRIVEN DEVELOPMENT FOR EARLY ASPECTS

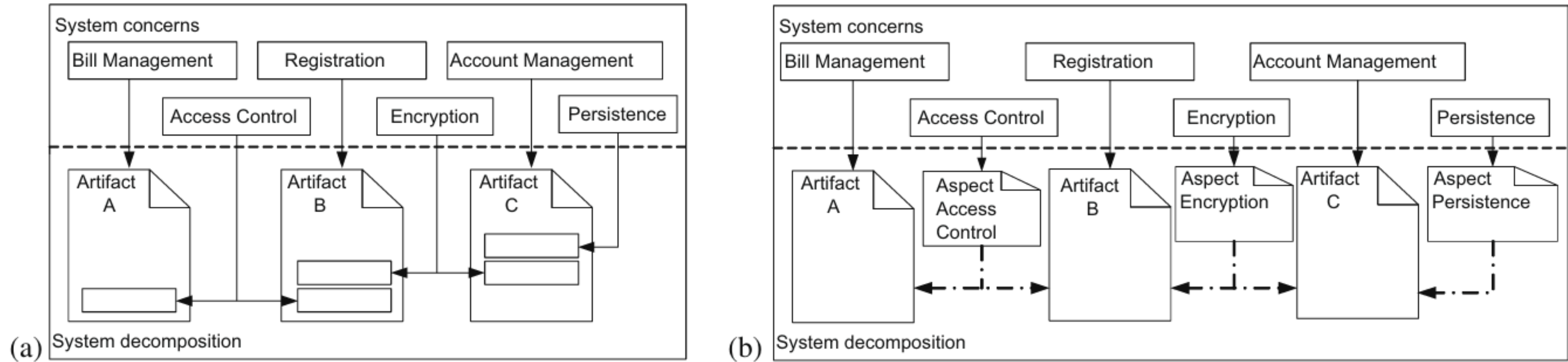
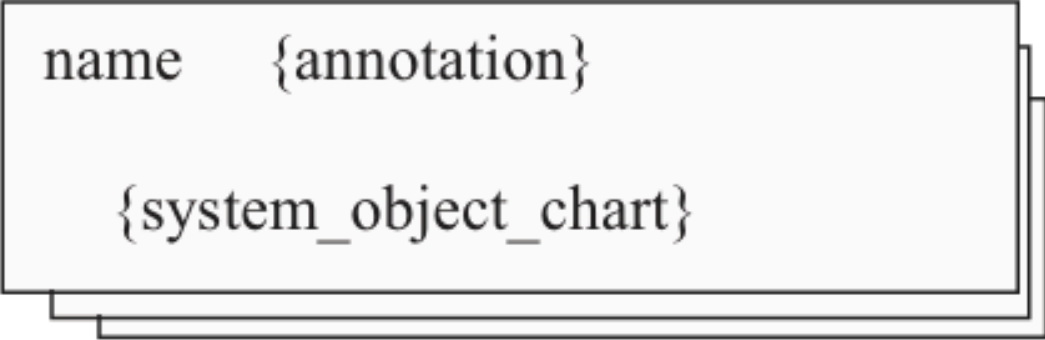
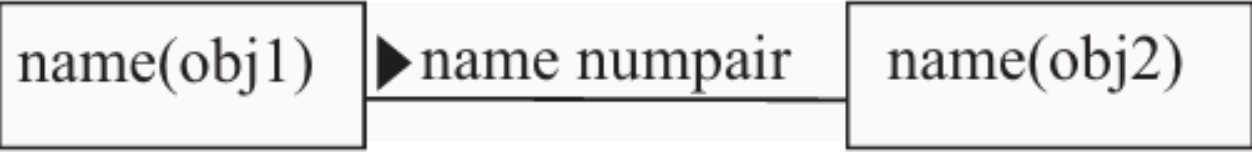


Fig. 1. Classical software decomposition (left); modularisation of crosscutting concerns (right).

- The paper combines Model-Driven Development with Aspect-Oriented Software Development
- The approach automates transformations between software models and architectures
- Non-functional requirements are modeled using reusable aspect-oriented patterns
- The work demonstrates systematic model transformation techniques

RIGOROUS EBNF-BASED DEFINITION FOR A GRAPHIC MODELING LANGUAGE

Graphic element	Equivalent textual element
	object name end
...	...
	object_set name {annotation} [contains {system_object_chart}] end
...	...
	association name numpair from obj1:object to obj2:object end
	state name
	start_state
...	...

- The paper proposes a formal EBNF-based syntax definition for graphical models
- Graphical elements are transformed into equivalent textual representations
- The approach enables parser-based syntax validation and processing
- Similar ideas are used in our ANTLR-based SQD parsing pipeline

AN EMPIRICAL EVALUATION OF LEX/YACC AND ANTLR PARSER GENERATION TOOLS





Welcome to the ANTLR lab, where you can learn about [ANTLR](#) or experiment with and test grammars! Just hit the [Run](#) button to try out the sample grammar.

To start developing with ANTLR, see [getting started](#).

[Feedback/issues](#) welcome. Brought to you by [Terence Parr](#), the maniac behind ANTLR.

Lexer [Parser](#) ?

```
1 parser grammar ExprParser;
2 options { tokenVocab=ExprLexer; }
3
4 program
5     : stat EOF
6     | def EOF
7     ;
8
9 stat: ID '=' expr ';'
10      | expr ';'
11      ;
12
13 def : ID '(' ID (',' ID)* ')' '{' stat* '}' ;
14
15 expr: ID
16      | INT
17      | func
18      | 'not' expr
19      | expr 'and' expr
20      | expr 'or' expr
21      ;
22
23 func : ID '(' expr (',' expr)* ')' ;
```

Input ?

```
1 f(x,y) {
2     a = 3+foo;
3     x and y;
4 }
```

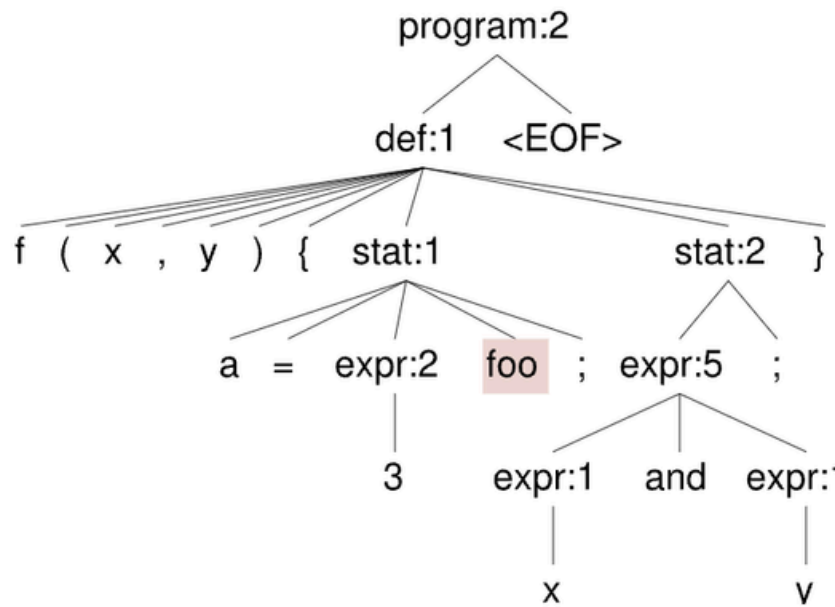
Start rule ?

[Run](#) [Show profiler](#)

Parser console

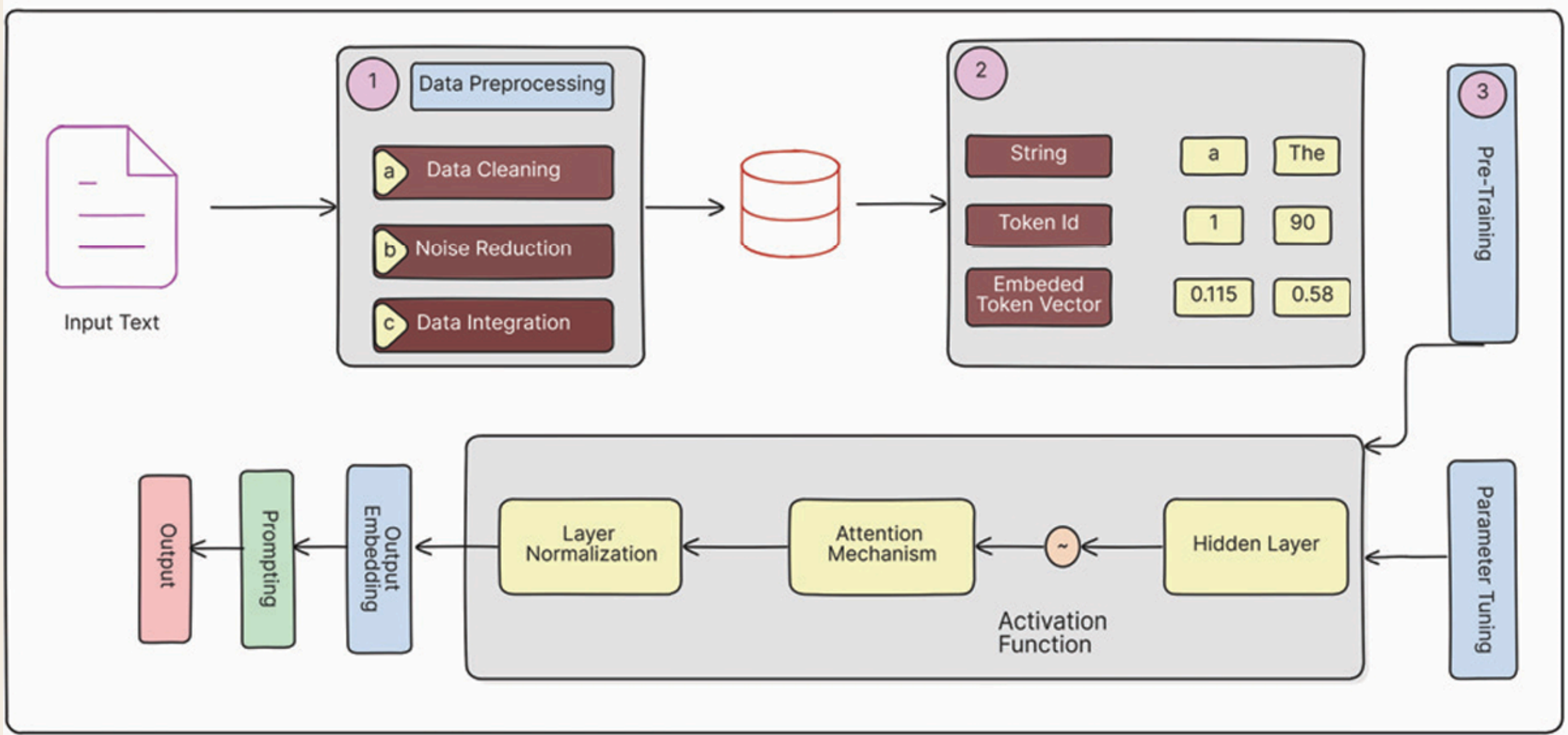
```
2:9 token recognition error at: '+'
2:10 extraneous input 'foo' expecting ';'

Tree Hierarchy
```



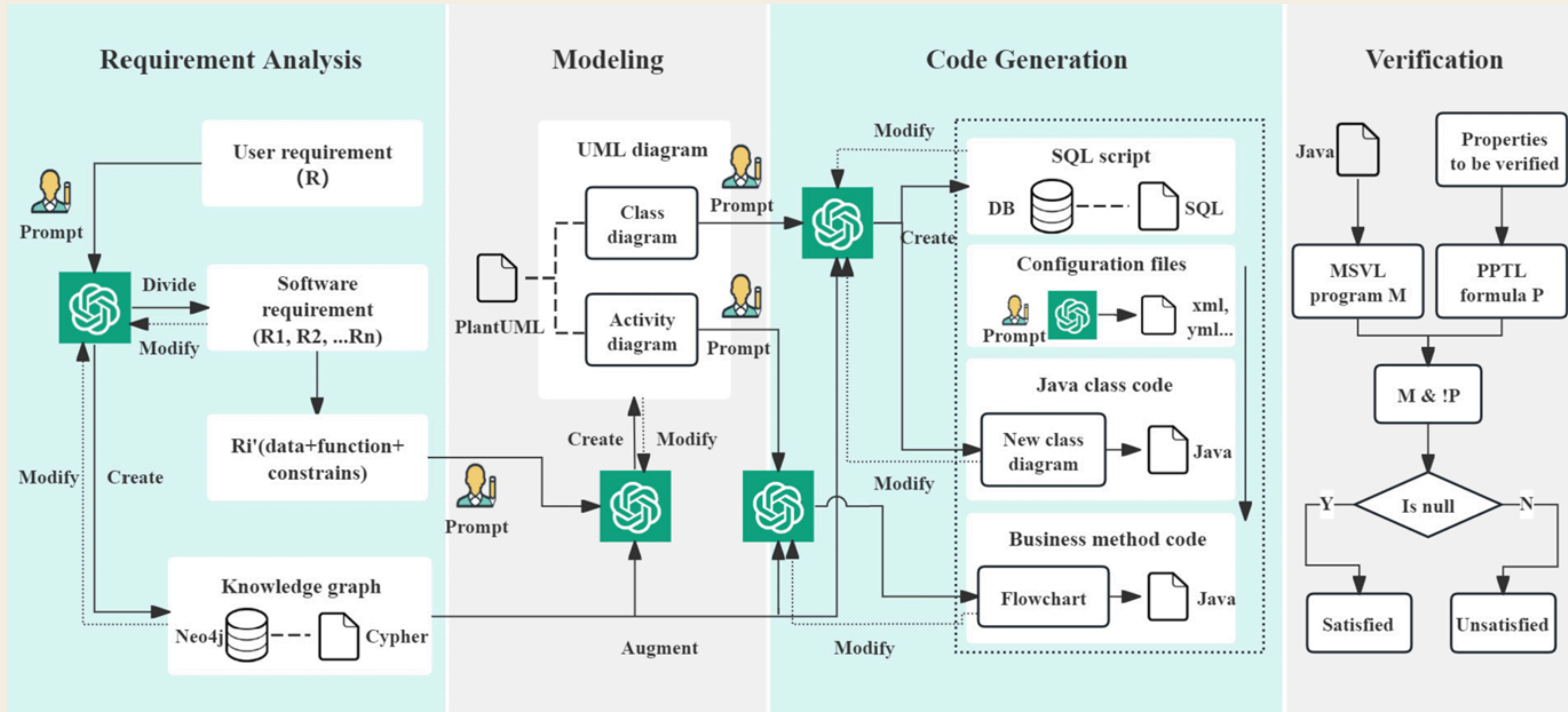
- The paper compares traditional parser generators with ANTLR-based parsing approaches
- ANTLR provides grammar readability, automatic parse tree generation, and visitor-based processing
- The study evaluates parser maintainability and development complexity
- The paper is relevant because our sequence diagram parser is implemented using ANTLR

EFFICIENT SOFTWARE DEVELOPMENT WITH UML-BASED CODE GENERATION



- The paper proposes an AI-assisted approach for generating Java code using ChatGPT
- UML diagrams are generated from requirements and later transformed into source code
- The process includes modeling, code generation, and verification phases
- The paper is relevant as an example of LLM-based code generation

GENERATING JAVA CODE PAIRING WITH CHATGPT

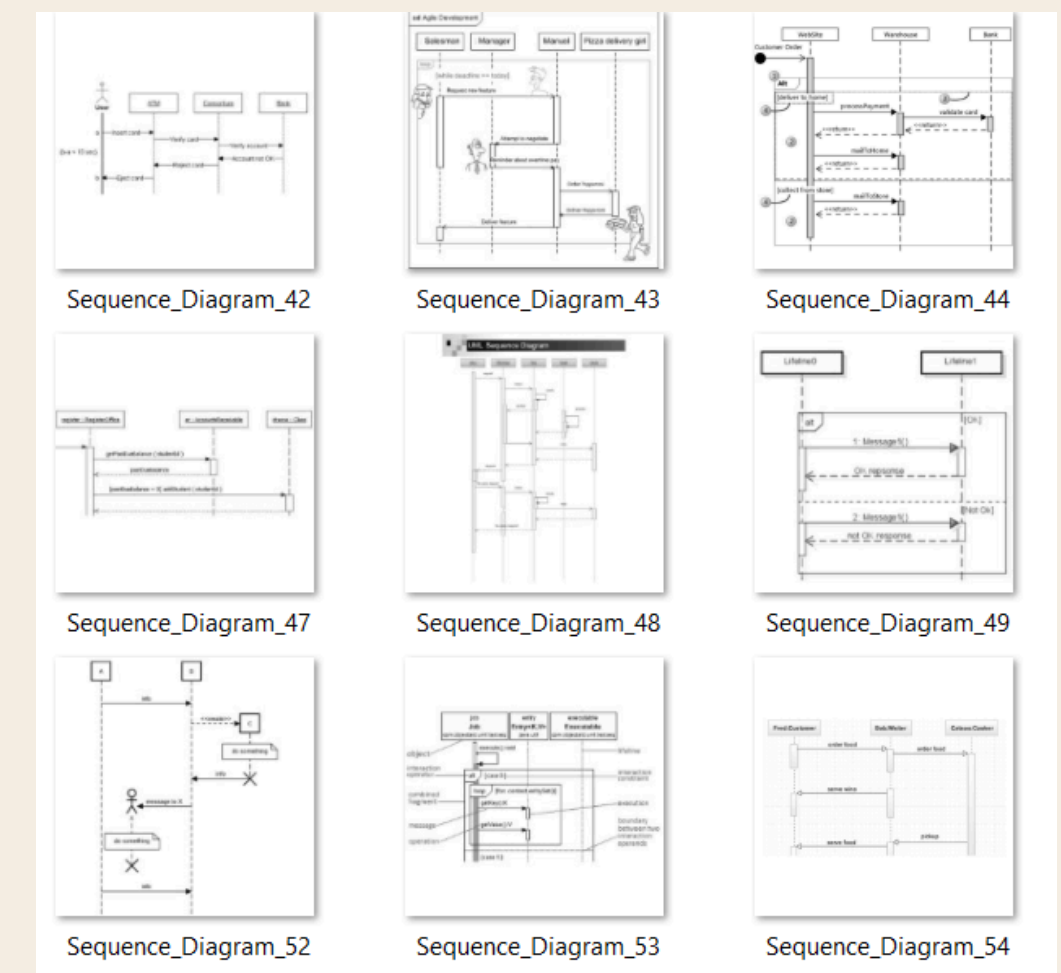


- The paper proposes AI-assisted Java code generation using ChatGPT
- The generated UML models are later transformed into Java source code
- The paper represents an LLM-based alternative to parser-based generation

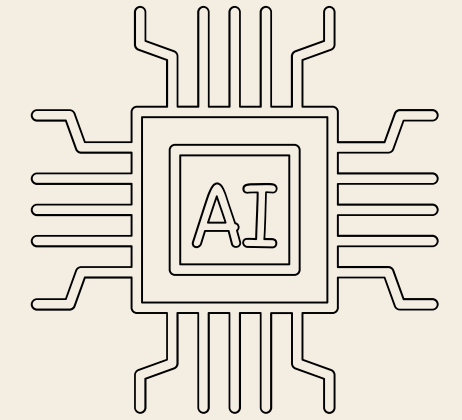
UML DATASETS



- Two UML datasets were selected for experiments and parser testing:
 - Dataset for Classification of UML Diagrams contains multiple UML diagram categories collected for software engineering education
 - Synthetic UML Diagram Dataset (PlantUML) contains large-scale synthetic UML diagrams generated using PlantUML
- The datasets are currently used for obtaining UML diagram examples and XML-based parser testing
- In future work, the datasets may be used for training and evaluation of AI-based UML generation systems



OUR APPROACH



Procedural parsing is used
for UML class diagrams

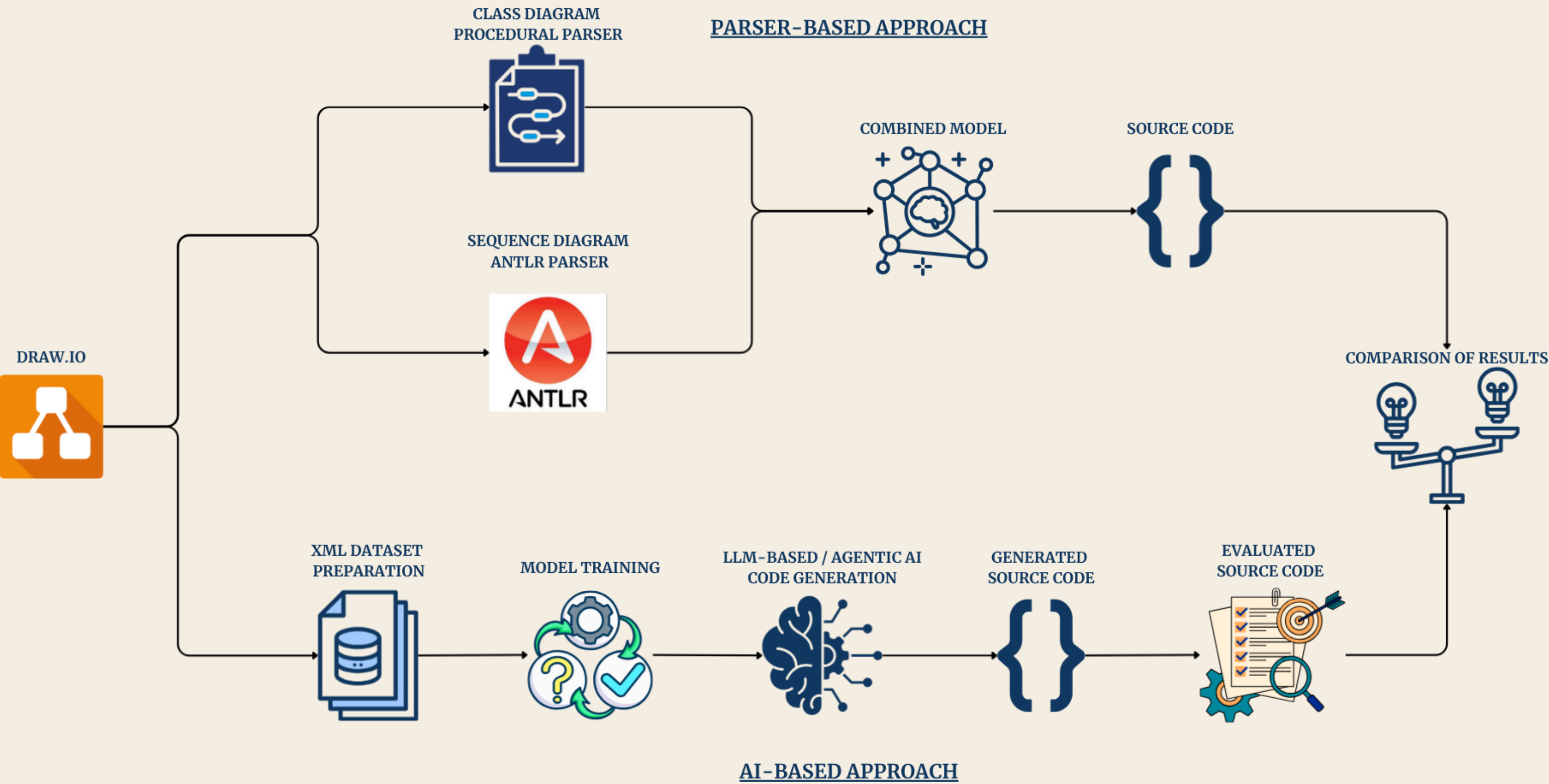
UML diagrams are
transformed into intermediate
representations

Compare parser-
based and AI-assisted
code generation
approaches

ANTLR grammar-based
parsing is used for UML
sequence diagrams

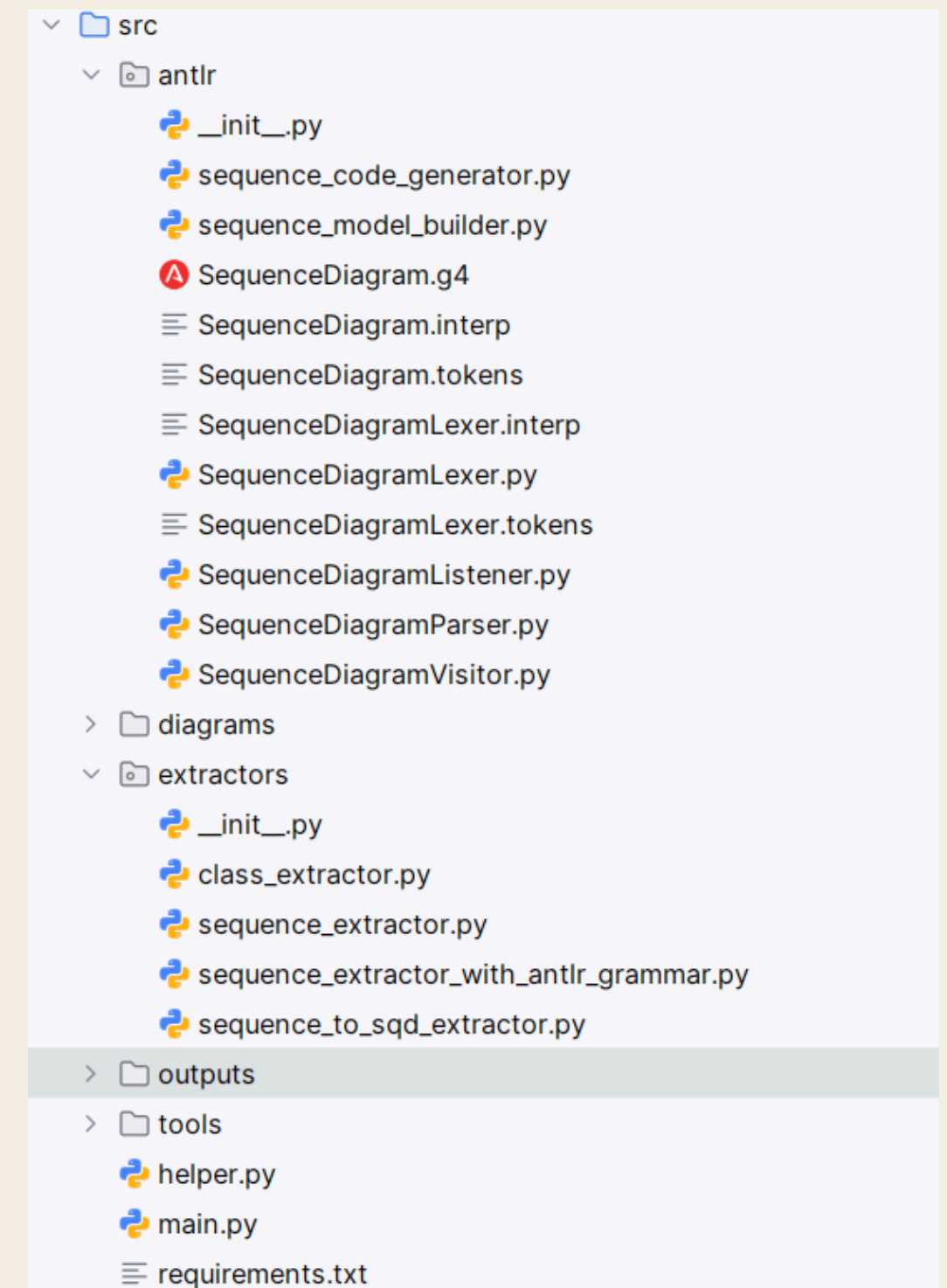
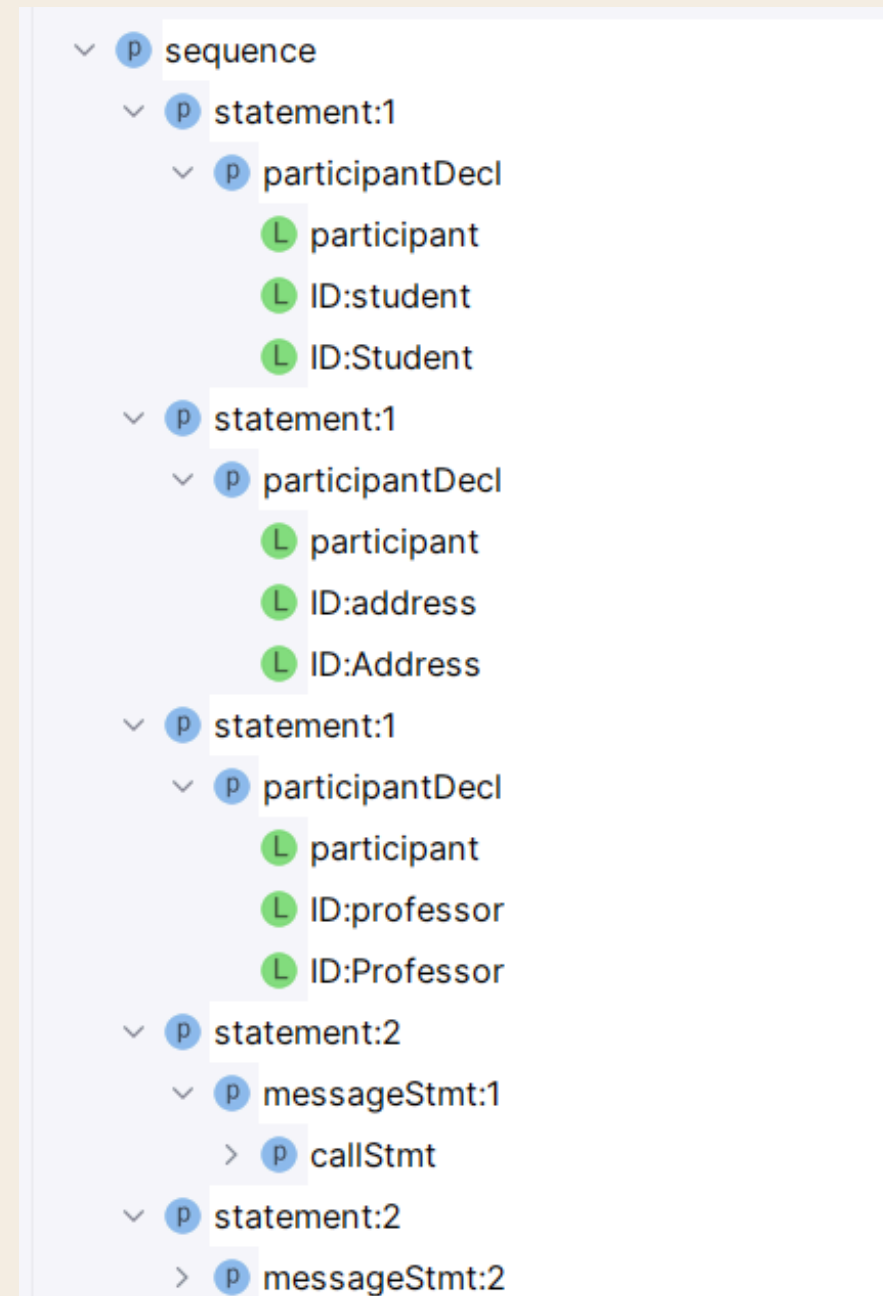
Parsed semantic models are
converted into Python source
code

SYSTEM ARCHITECTURE

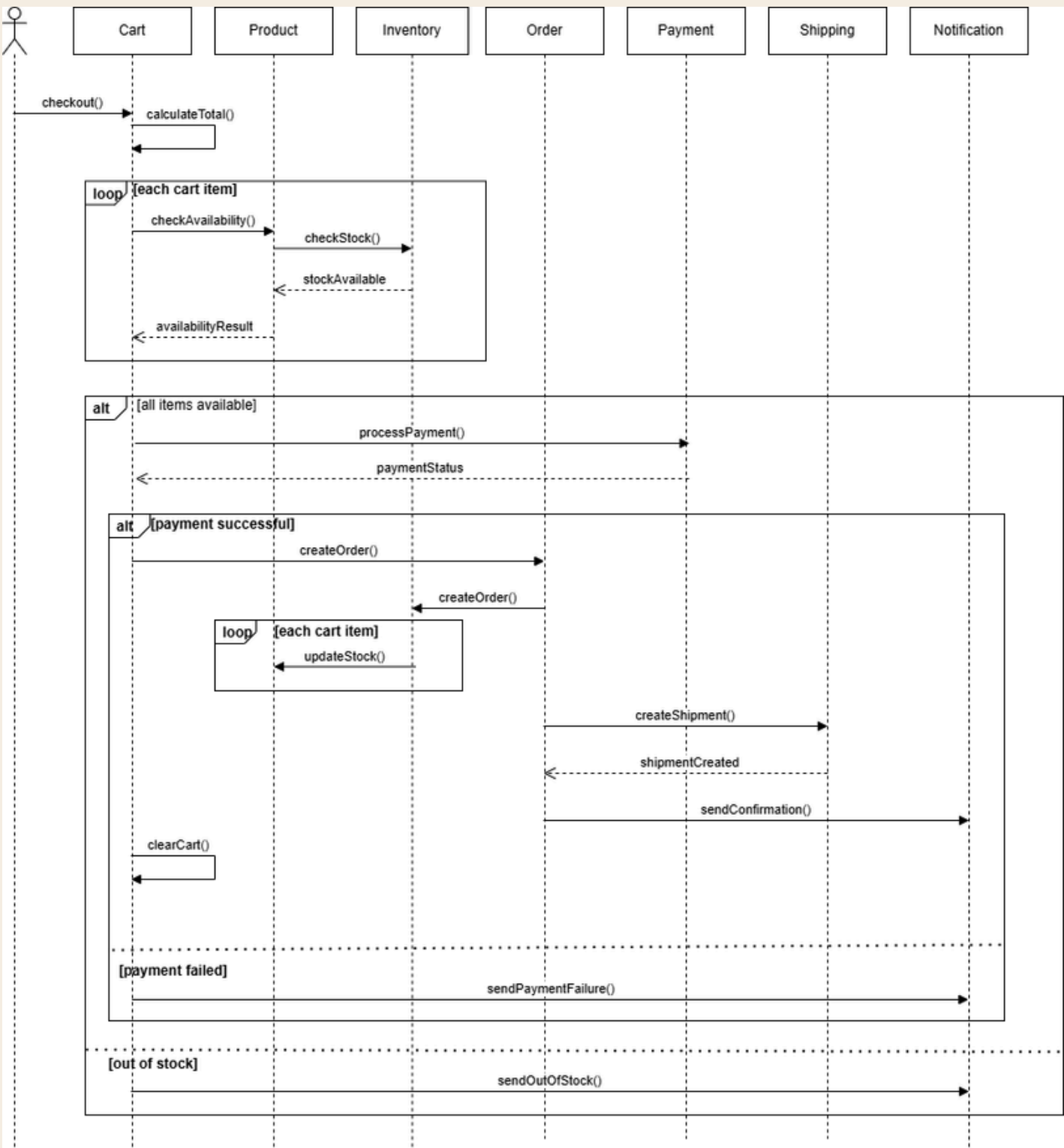
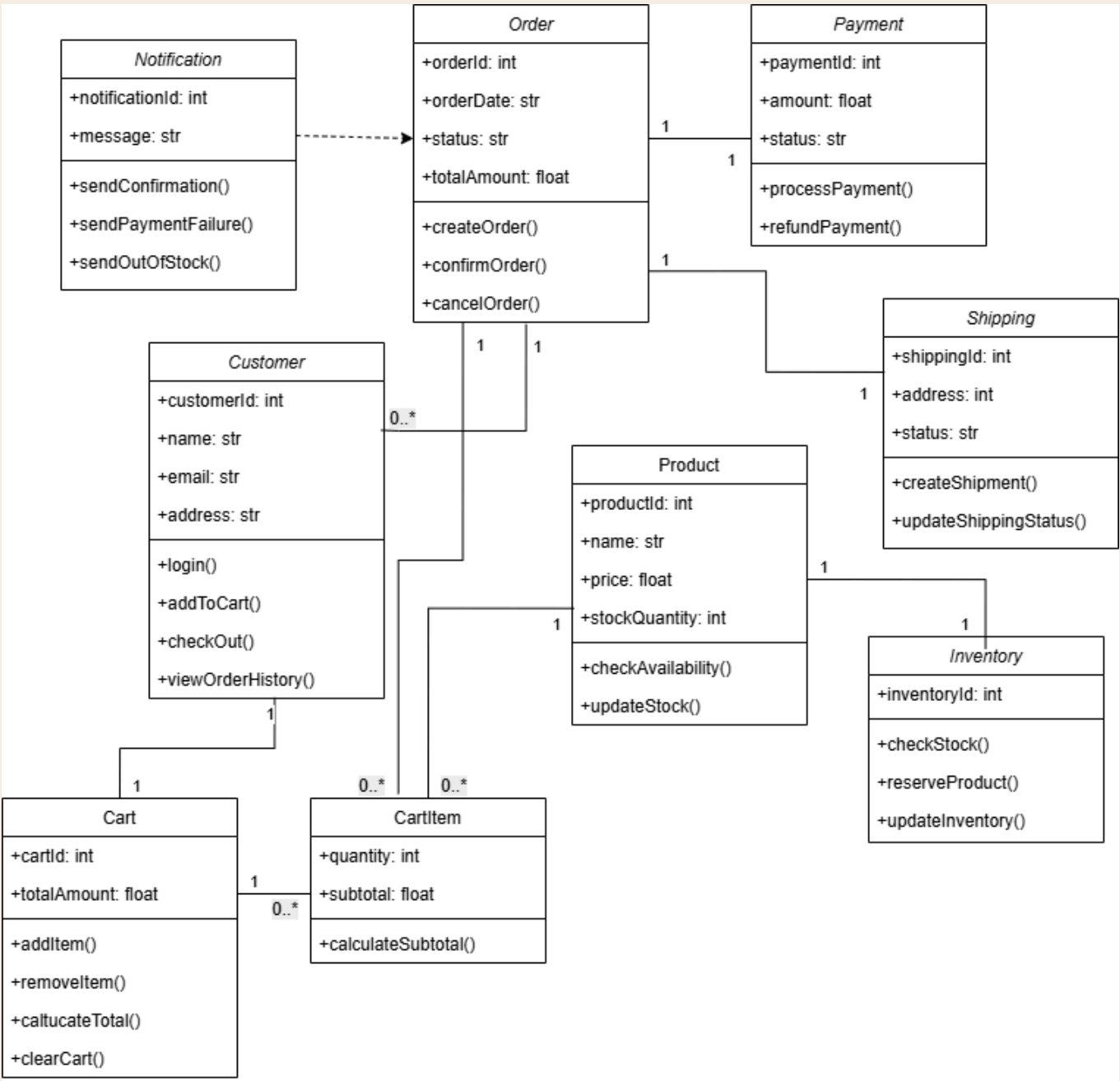


CURRENT PROGRESS

- Implemented XML extraction from draw.io diagrams
- Implemented procedural parser for UML class diagrams
- Designed intermediate SQD language for sequence diagrams
- Designed ANTLR grammar and visitor-based parsing
- Implemented Python code generation from parsed UML models
- Added support for calls, self-calls, alt, and loop fragments



EXPERIMENTAL RESULTS



- Created class and Sequence Diagrams for experiment

EXPERIMENTAL RESULTS

```
1 class Customer: 1 usage  ⚡ Jahon27
2     def __init__(self):  ⚡ Jahon27
3         self.customerId = None
4         self.name = None
5         self.email = None
6         self.address = None
7
8     def login(self):  ⚡ Jahon27
9         pass
10
11     def addToCart(self):  ⚡ Jahon27
12         pass
13
14     def checkOut(self):  ⚡ Jahon27
15         pass
16
17     def viewOrderHistory(self):  ⚡ Jahon27
18         pass
19
20 class Cart: 1 usage  ⚡ Jahon27
21     def __init__(self):  ⚡ Jahon27
22         self.cartId = None
23         self.totalAmount = None
24
25     def addItem(self):  ⚡ Jahon27
26         pass
```

- Generated Python Code

```
126  ✓ def run_sequence(): 1 usage  ⚡ Jahon27
127      customer = Customer()
128      cart = Cart()
129      product = Product()
130      inventory = Inventory()
131      order = Order()
132      payment = Payment()
133      shipping = Shipping()
134      notification = Notification()
135
136      condition = False
137      eachCartItem = False
138      paymentSuccessful = False
139
140      customer.checkOut()
141      cart.calculateTotal()
142  ✓ while condition:
143      product.checkAvailability()
144      inventory.checkStock()
145  ✓ if paymentSuccessful:
146      payment.processPayment()
147  ✓ if eachCartItem:
148      order.createOrder()
149  ✓ while condition:
150      product.updateStock()
151      shipping.createShipment()
152      notification.sendConfirmation()
153      cart.clearCart()
154      notification.sendPaymentFailure()
155  ✓ else:
156      notification.sendOutOfStock()
```

FUTURE WORK

- Improve semantic analysis and generated source code quality
- Extend support for more UML diagram types
- Compare parser-based generation with LLM-based code generation approaches
- Investigate AI-assisted UML interpretation and automatic business logic generation
- Use UML datasets for training and evaluation of AI models
- Evaluate the system on larger UML datasets



**THANK YOU
FOR YOUR ATTENTION!**